Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Сравнение функций потерь MSE и Binary Cross-Entropy для задачи XOR»

Выполнил:
Студент 4 курса
Группы ИИ-26
Турич Д. А.
Проверил:
Никитин С. К.

Брест 2026

Цель работы

Исследовать влияние функций потерь MSE и BCE на обучение MLP 2–2–1 для XOR с классами −9 и 6, сравнить сходимость, устойчивость, точность восстановления числовой шкалы и разделяющую поверхность.

1. Вариант 6 и постановка задачи

A	B	Ожидается
-9	-9	-9
-9	6	6
6	-9	6
6	6	-9

2. Архитектура, масштабирование и градиент BCE

Сеть 2–2–1 имеет сигмоиду на обоих слоях; обучение — online backpropagation, $\eta=0{,}5$, максимум 20 000 эпох, случайные веса $U(-0{,}1;0{,}1)$, $E_e=0{,}01$. Во всех запусках входы нормализованы $x_n=(x+9)/15$.

Конфигурация А: на выходе сигмоида и MSE по исходным −9/6 — дословный базовый вариант методички. Конфигурация Б: BCE на бинарных метках $t_n=(t+9)/15 \in \{0,1\}$; результат переводится в исходную шкалу формулой $y_r=-9+15\hat{y}_n$. Для контроля показана нормализованная MSE-модель (отдельный эксперимент).

$BCE = -\Sigma[t \ln(p)+(1-t)\ln(1-p)]$; при sigmoid $p=\sigma(z)$ градиент BCE по логиту упрощается до $\partial L/\partial z=p-t$. Для скрытого слоя $\delta_1=(\delta_2 W_2^T)\odot h(1-h)$. Для MSE выходной $\delta_2=(p-t)p(1-p)$.

Внимание: значения порога $E_e=0{,}01$ для MSE и BCE численно неэквивалентны — это потери с различными размерностями и шкалами. В отчёте сравнение качества проводится преимущественно по accuracy и MAE исходной шкалы, а не по абсолютным величинам различных loss.

3. Численные результаты (5 seed на конфигурацию)

А — исходная MSE

Seed	Эпохи	E_s	Accuracy	MAE	Сходимость
6	20000	117.00004	50%	7.5000	нет
7	20000	117.00004	50%	7.5000	нет
11	20000	117.00003	50%	7.5000	нет
14	20000	117.00004	50%	7.5000	нет
18	20000	117.00004	50%	7.5000	нет

Успешных запусков: 0 из 5. Ни один запуск не достиг E_e.

Б — BCE

Seed	Эпохи	E_s	Accuracy	MAE	Сходимость
6	2368	0.00999	100%	0.0374	да

Seed	Эпохи	E_s	Accuracy	MAE	Сходимость
7	2792	0.01000	100%	0.0374	да
11	20000	1.91012	75%	4.9746	нет
14	2523	0.01000	100%	0.0374	да
18	9647	0.01000	100%	0.0374	да

Успешных запусков: 4 из 5. Для сошедшихся: диапазон эпох 2368–9647.

Диагностика — MSE после нормализации

Seed	Эпохи	E_s	Accuracy	MAE	Сходимость
6	2769	0.01000	100%	1.0467	да
7	2931	0.00999	100%	1.0458	да
11	2810	0.01000	100%	1.0462	да
14	2866	0.00999	100%	1.0463	да
18	3244	0.00999	100%	1.0450	да

Успешных запусков: 5 из 5. Для сошедшихся: диапазон эпох 2769–3244.

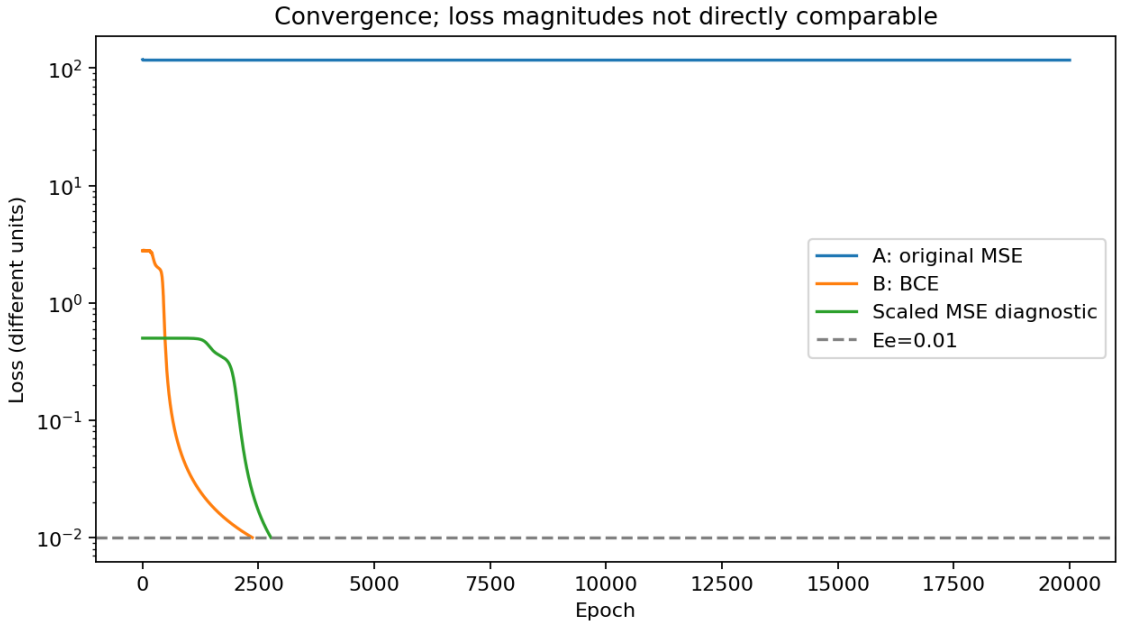


Рисунок 1 — Кривые обучения MSE и BCE

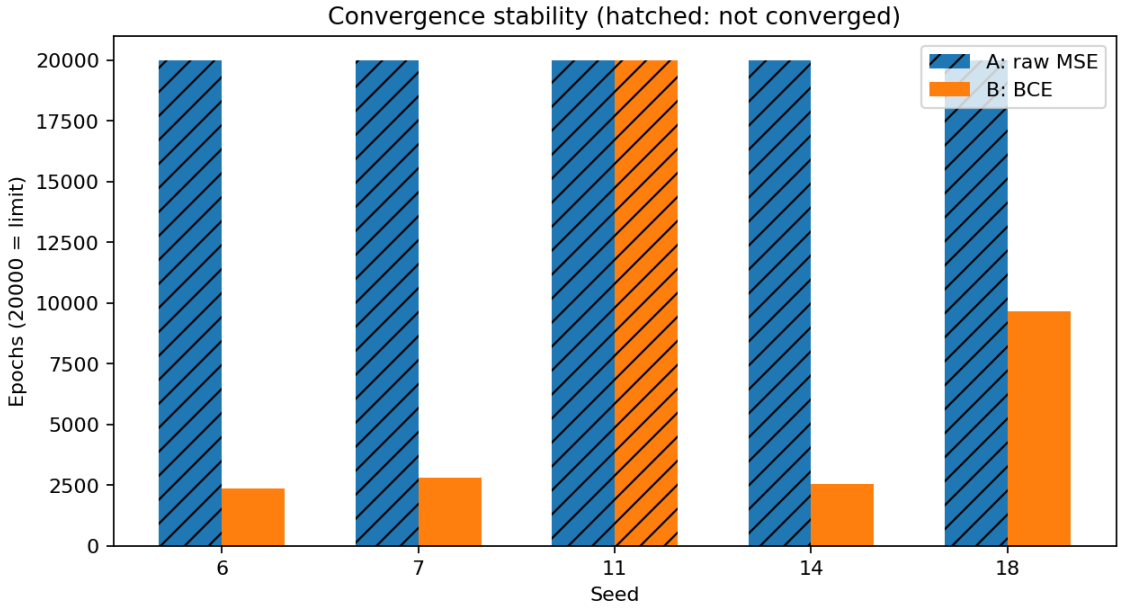


Рисунок 2 — Устойчивость по пяти случайным инициализациям

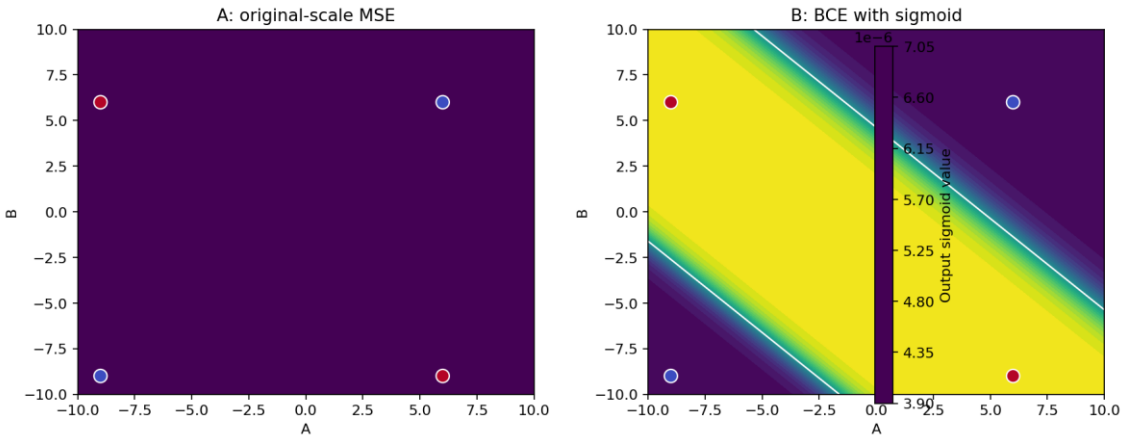


Рисунок 3 — Карты выхода сигмоиды на плоскости A, B; точки — XOR

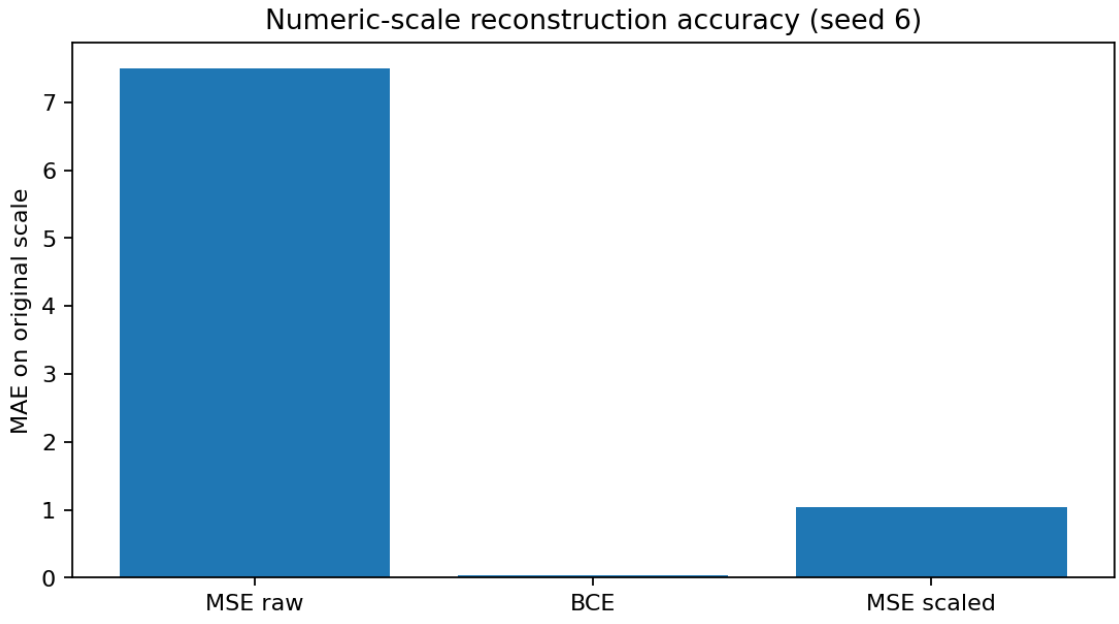


Рисунок 4 — Сравнение MAE после перевода в исходную шкалу

4. Режим функционирования

A	B	$\hat{y}_n$	$\hat{y}_r$	Класс
-9	-9	0.0021	-8.9684	-9
-9	6	0.9978	5.9664	6
6	-9	0.9978	5.9664	6
6	6	0.0034	-8.9491	-9
-6	4	0.9972	5.9574	6
0	0	0.9927	5.8910	6
6	0	0.1514	-6.7291	-9
-10	10	0.9927	5.8910	6

5. Выводы по вопросам методички

1) Сравнение буквальной MSE с BCE отражает одновременно смену потерь и обязательную нормализацию шкалы; поэтому делать чистый вывод о преимуществе функции потерь нельзя. BCE после нормализации достигла E_e в 4 из 5 запусков и accuracy 100% в успешных запусках.

2) Нормализация сделала задачу численно допустимой для сигмоиды: значения −9/6 преобразованы в 0/1. Прямая MSE этого обеспечить не могла.

3) Для успешного запуска BCE ошибка восстановления исходной шкалы составляет примерно 0,0374, тогда как прямая MSE даёт около 7,5. Корректнее сопоставлять BCE с нормализованной MSE: у неё на пороге 0,01 MAE около 1,05.

4) Случайная инициализация значимо влияет на скорость и результат: BCE в одном из пяти запусков не достигла E_e. Наблюдаемая стагнация означает плохую стационарную область, но сама по себе не доказывает строгий локальный минимум.

5) Выбор кодов −9 и 6 не устраняет линейную неразделимость XOR; она определяется взаимным расположением четырёх точек.

6) BCE может улучшить градиент при сигмоиде, но не заменяет необходимый скрытый слой в стандартной архитектуре из линейных комбинаций и монотонных активаций.

7) В нормализованной BCE-карте видна нелинейная граница между двумя областями противоположных классов. Буквальная MSE по исходным целям вырождена из-за невозможной шкалы выхода; сравнивать её контур напрямую как равноправную обученную классификацию нельзя.

Листинг программы (полный исходный код)

В архиве код также сохранён отдельными редактируемыми файлами Python; ниже приведён полный листинг без скрытых библиотек машинного обучения.

Файл: xor_engine.py

```
001: """MLP 2-2-1 and single layer perceptron. NumPy is the only dependency.
002: All losses are totals over the four training examples, evaluated after an epoch.
003: """
004: import numpy as np
005:
006: C0, C1 = -9.0, 6.0
```

```
X_RAW = np.array([[-9.,-9.],[-9.,6.],[6.,-9.],[6.,6.]])
Y_RAW = np.array([-9.,6.,6.,-9.])
X = (X_RAW - C0) / (C1-C0)
Y = (Y_RAW - C0) / (C1-C0)

def sigmoid(z):
    z = np.clip(z, -60, 60)
    return 1 / (1+np.exp(-z))

def relu(z):
    return np.maximum(z, 0.)

class MLP:
    def __init__(self, seed=7, activation='sigmoid', loss='bce', lr=1.0):
        rng=np.random.RandomState(seed)
        self.w1=rng.uniform(-.1,.1,(2,2))
        self.b1=rng.uniform(-.1,.1,2)
        self.w2=rng.uniform(-.1,.1,2)
        self.b2=float(rng.uniform(-.1,.1))
        self.activation,self.loss,self.lr=activation,loss,lr

    def forward(self, x):
        z1=x@self.w1+self.b1
        h=sigmoid(z1) if self.activation=='sigmoid' else relu(z1)
        p=sigmoid(h@self.w2+self.b2)
        return z1,h,p

    def predict(self, x):
        return np.asarray(self.forward(np.asarray(x, dtype=float))[2])

    def train_step(self,x,t):
        z,h,p=self.forward(x)
        # BCE + sigmoid: dL/dz = p-t; MSE=1/2*(p-t)^2.
        delta=(p-t) if self.loss=='bce' else (p-t)*p*(1-p)
        dh=(self.w2*delta)*(h*(1-h) if self.activation=='sigmoid' else (z>0))
        old_w2=self.w2.copy()
        self.w2-=self.lr*delta*h
        self.b2-=self.lr*delta
        self.w1-=self.lr*np.outer(x,dh)
        self.b1-=self.lr*dh

    def total_loss(self, xx, yy):
        p=np.clip(self.predict(xx),1e-12,1-1e-12)
        if self.loss=='bce': return float(-np.sum(yy*np.log(p)+(1-yy)*np.log1p(-p)))
        return float(.5*np.sum((p-yy)**2))

    def fit(self,xx,yy,max_epochs=20000,ee=.01,shuffle=True):
        rng=np.random.RandomState(42)
        history=[]
        for epoch in range(1,max_epochs+1):
            order=rng.permutation(len(xx)) if shuffle else range(len(xx))
            for i in order:self.train_step(xx[i],yy[i])
            error=self.total_loss(xx,yy)
            history.append(error)
            if error<=ee:break
        return dict(epochs=epoch,Es=error,converged=error<=ee,history=np.asarray(history))

class SinglePerceptron:
    def __init__(self,seed=7,lr=1.):
        rng=np.random.RandomState(seed)
        self.w=rng.uniform(-.1,.1,2)
        self.b=float(rng.uniform(-.1,.1))
        self.lr=lr
    def predict(self,xx):return sigmoid(np.asarray(xx)@self.w+self.b)
    def fit(self,xx,yy,max_epochs=20000,ee=.01):
        rng=np.random.RandomState(42);h=[]
        for ep in range(1,max_epochs+1):
            for i in rng.permutation(len(xx)):
                x,t=xx[i],yy[i]
                p=float(self.predict(x)); delta=(p-t)*p*(1-p)
                self.w-=self.lr*delta*x;self.b-=self.lr*delta
            p=self.predict(xx);es=float(.5*np.sum((p-yy)**2));h.append(es)
            if es<=ee:break
        return dict(epochs=ep,Es=es,converged=es<=ee,history=np.asarray(h))

def stats(model, raw_output=False):
    out=model.predict(X)
    if raw_output:
        yr=out;cls=np.where(abs(yr-C0)<=abs(yr-C1),C0,C1)
    else:
```

```
087:         yr=C0+(C1-C0)*out;cls=np.where(out<.5,C0,C1)
088:     return {'accuracy':float(np.mean(cls==Y_RAW)), 'mae':float(np.mean(abs(yr-Y_RAW))),
089:             'pred_raw':np.asarray(yr).tolist(),'pred_norm':np.asarray(out).tolist(),
090:             'pred_class':np.asarray(cls).tolist()}
091:
092: def inference(model,a,b,raw_output=False):
093:     if not (-10<=a<=10 and -10<=b<=10):raise ValueError('A and B must be in [-10,10]')
094:     pn=float(model.predict((np.array([a,b])-C0)/(C1-C0)))
095:     val=pn if raw_output else C0+15*pn
096:     cls=C0 if abs(val-C0)<=abs(val-C1) else C1
097:     return {'A':a,'B':b,'output_norm':pn,'output_raw':val,'class':cls}
098:
099: TEST_PAIRS=[(-9,-9),(-9,6),(6,-9),(6,6),(-6,4),(0,0),(6,0),(-10,10)]
```

Файл: main.py

```
001: """Lab 2 (variant 6). Compare original-scale MSE with BCE and scaled MSE."""
002: import argparse,json
003: from pathlib import Path
004: import numpy as np
005: import matplotlib.pyplot as plt
006: from xor_engine import MLP, X,Y,Y_RAW,C0,C1,stats,inference,TEST_PAIRS
007: SEEDS=[6,7,11,14,18]
008:
009: def fit_many(name,loss,targets,max_epochs):
010:     runs=[]
011:     for seed in SEEDS:
012:         model=MLP(seed=seed,loss=loss,lr=.5)
013:         r=model.fit(X,targets,ee=.01,max_epochs=max_epochs)
014:         s=stats(model,raw_output=(name=='raw MSE'))
015:         print(name,seed,r['epochs'],r['Es'],s['accuracy'],s['mae'])
016:         runs.append((model,r,s))
017:     return runs
018:
019: def plot_results(results,path):
020:     plt.figure(figsize=(8,5))
021:     for key in ('raw MSE','BCE','scaled MSE diagnostic'):
022:         h=results[key][0][1]['history']
023:         plt.plot(range(1,len(h)+1),h,label=key)
024:     plt.axhline(.01,linestyle='--',label='Ee=0.01')
025:     plt.yscale('log');plt.xlabel('Epoch');plt.ylabel('Total training loss')
026:     plt.title('Convergence, variant 6');plt.legend();plt.tight_layout()
027:     plt.savefig(path,dpi=160);plt.close()
028:
029: def main(max_epochs=20000,interactive=False):
030:     results={'raw MSE':fit_many('raw MSE','mse',Y_RAW,max_epochs),
031:              'BCE':fit_many('BCE','bce',Y,max_epochs),
032:              'scaled MSE diagnostic':fit_many('scaled MSE diagnostic','mse',Y,max_epochs)}
033:     outdir=Path(__file__).parent/'results';outdir.mkdir(exist_ok=True)
034:     plot_results(results,outdir/'convergence_runtime.png')
035:     summary={name:[{'seed':SEEDS[i],'epochs':r['epochs'],'Es':r['Es'],
036:       'converged':r['converged'],**s} for i,(_,r,s) in enumerate(runs)]
037:       for name,runs in results.items()}
038:     with open(outdir/'latest_run.json','w',encoding='utf8') as f:
039:         json.dump(summary,f,ensure_ascii=False,indent=2)
040:     if interactive:
041:         model=results['BCE'][0][0]
042:         while True:
043:             a=input('A (or q): ').strip()
044:             if a.lower()=='q':break
045:             try:print(inference(model,float(a),float(input('B: '))))
046:             except ValueError as e:print(e)
047:
048: if __name__=='__main__':
049:     ap=argparse.ArgumentParser();ap.add_argument('--
quick',action='store_true');ap.add_argument('--interactive',action='store_true')
050:     args=ap.parse_args();main(1000 if args.quick else 20000,args.interactive)
```